HealthTrack

PROJECT REPORT, PHASE 1 (FRONTEND)

HealthTrack

Personal Fitness and Wellness Tracker

COURSE
CSCI426 Advanced Web Programming

PROJECT TITLE
HealthTrack

PHASE
1: React frontend

STUDENT
Maroun Issa

STUDENT ID
52430635

INSTRUCTOR
Ihab Hassoun

GITHUB REPOSITORY
github.com/scott-pilgrim-lab/healthtrack

LIVE SITE
healthtrack-henna.vercel.app

Contents

1 Abstract

2 Problem and objectives

3 System design

4 Technologies used

5 Design

6 Key code snippets

7 Testing

8 Phase 2 plan

9 Conclusion

10 Group contribution statement

11 References

1. Abstract

HealthTrack is a single-page web application that helps a person keep track of three daily health habits: physical activities, meals and workouts. The user records each item with a short form, and the application calculates a summary of the current day (calories eaten, calories burned, workout minutes and number of entries) and a comparison of the last seven days.

Phase 1 delivers the complete frontend. It is built with React and Vite, uses React Router for navigation between six routes, and uses Bootstrap 5 through the react-bootstrap package for the grid and the interface components. A single custom stylesheet overrides Bootstrap's CSS variables to give the site its own visual identity. Data is saved in the browser with `localStorage` behind a small service file, so that Phase 2 can replace it with calls to a backend API and a database without changing the pages or components.

2. Problem and objectives

People who try to follow their fitness usually keep the information in several places: workouts in a notes app, meals in memory and activity in a third app. Nothing shows the whole day, and large fitness platforms ask for an account and many fields before a single meal can be logged.

The objectives of HealthTrack are:

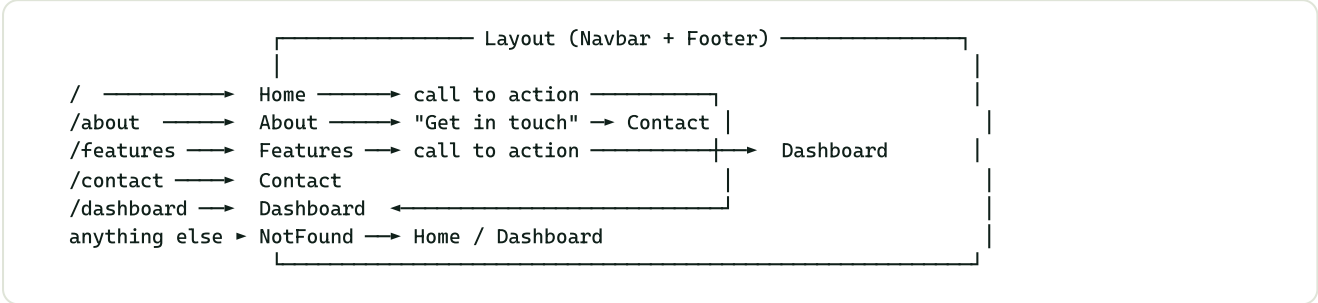
1. Provide one place to log activities, meals and workouts quickly.
2. Show a clear summary of the day that updates immediately.
3. Show a simple view of the last seven days.
4. Work well on phones, tablets and desktops.
5. Be structured so a backend and database can be added in Phase 2.

3. System design

3.1 Page map

Route	Page	Content
`/`	Home	Hero section, short pitch, three feature highlights, three steps, call-to-action to the Dashboard
`/about`	About	Purpose of the app, the problem it solves, about the developer
`/features`	Features	Cards for the three trackers, the daily summary and extra features
`/contact`	Contact	Form (name, email, message) with validation and a success state

Route	Page	Content
`/dashboard`	Dashboard	Summary cards, 7-day chart, daily goals, tabs for the three trackers
`*`	NotFound	404 page for any unknown address



3.2 Component tree

```
main.jsx
└── BrowserRouter
    └── App                      (route table)
        └── Layout               (shared shell)
            ├── AppNavbar
            │   └── Brand
            ├── <Outlet/>        (the current page)
            │   ├── Home
            │   │   └── FeatureCard ×3
            │   ├── About
            │   │   ├── PageHeader
            │   │   └── FeatureCard ×3
            │   ├── Features
            │   │   ├── PageHeader
            │   │   └── FeatureCard ×7
            │   ├── Contact
            │   │   └── PageHeader
            │   ├── Dashboard
            │   │   ├── PageHeader
            │   │   ├── StatCard ×4
            │   │   ├── WeeklyChart
            │   │   ├── GoalProgress
            │   │   └── Tabs
            │   │       └── TrackerPanel ×3   (Activities, Meals, Workouts)
            │   │           ├── EntryForm
            │   │           ├── DateFilter
            │   │           ├── EntryList
            │   │           └── EmptyState
            │   └── NotFound
            └── Footer
                └── Brand
```

`Icon` is a small helper used by most of these components.

3.3 Folder structure

Folder	Responsibility
`src/pages`	One component per route

Folder	Responsibility
`src/components`	Reusable interface pieces that receive data through props
`src/services`	`storage.js`, the only file that reads or writes data
`src/data`	Static configuration: tracker fields, sample data, navigation links
`src/utils`	Pure helper functions for dates and totals
`src/styles`	`theme.css`, the single custom stylesheet

3.4 Data model

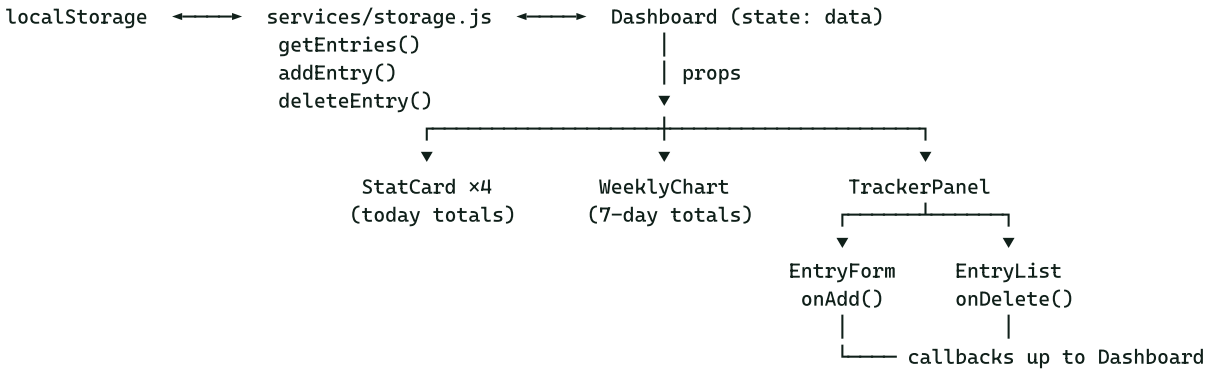
Every entry is a plain object. All three types share `id`, `date` (stored as `YYYY-MM-DD`) and `name`.

Type	Fields
Activity	`id`, `date`, `name`, `duration` (min), `calories` (burned)
Meal	`id`, `date`, `name`, `category` (Breakfast, Lunch, Dinner, Snack), `calories`, `protein` (g, optional)
Workout	`id`, `date`, `name`, `category` (Push, Pull, Legs, Skills, Full body), `duration` (min), `calories` (burned), `notes` (exercises, optional)

In Phase 2 each type maps directly to one database table and one group of API endpoints.

3.5 Data flow

The Dashboard page owns all the entries in one piece of state. Data moves in one direction:



1. When the Dashboard loads, it calls `getEntries()` for each type and stores the result in state.
2. The summary cards, the chart and the goals are **calculated** from that state on every render (`utils/stats.js`). They are not stored separately, so they always agree with the lists.
3. When the user submits a form, `EntryForm` validates the values and calls the `onAdd` callback it received through props. The Dashboard saves the entry with `addEntry()` and then updates its state.

4. React re-renders, and the cards, chart and list all show the new entry.
5. Deleting works the same way through `onDelete` and `deleteEntry()`.

The date filter is local state inside each `TrackerPanel`, because it only affects that panel's list.

4. Technologies used

Technology	Role in the project
React 19 (JavaScript)	Function components, `useState` and `useEffect` hooks, props
Vite	Development server with hot reload and the production build
React Router	`BrowserRouter`, `Routes`, nested routes with `Outlet`, `NavLink` for active links, catch-all route for the 404 page
Bootstrap 5 + react-bootstrap	`Navbar`, `Container`, `Row`, `Col`, `Card`, `Form`, `Button`, `Tabs`, `ProgressBar`, `Alert` and the responsive grid
Custom CSS	`theme.css` overrides Bootstrap's CSS variables and adds the hero, stat cards, entry list and chart
localStorage	Saves entries in the browser during Phase 1
ESLint	Static checks for JavaScript and React hooks
Git and GitHub	Version control with small commits per feature
Vercel	Hosting, with a rewrite rule for client-side routes

5. Design

5.1 Visual identity

Element	Choice
Primary colours	Deep forest `#10201a` (dark surfaces, text), forest green `#17503a` (buttons, links)
Accent	Lime `#c8f03c`, used only on dark surfaces so the contrast stays high
Data colours	Amber `#f2a516` for calories eaten, coral `#f0603c` for calories burned
Background	Warm off-white `#f5f6ef` with white cards
Fonts	Space Grotesk for headings and numbers, DM Sans for body text
Shape	Rounded cards (1rem radius) and pill-shaped buttons and tabs

Element	Choice
Spacing scale	4, 8, 16, 24, 40, 64, 96 px (`--ht-space-1` to `--ht-space-7`)
Type scale	1.25 ratio from 0.8rem to a fluid 4rem heading

All of these are defined once as CSS variables at the top of `theme.css` and reused everywhere.

5.2 Responsiveness

The layout was checked at 375px, 768px and 1280px, and tested with phone emulation (touch input, high-density screen) at 320, 360, 375, 390 and 414px wide plus a phone in landscape. No page scrolls sideways at any of these sizes.

Width	Behaviour
375px (phone)	Hamburger menu, content in a single column, summary cards two per row, form above the list
768px (tablet)	Hamburger menu, feature cards in rows of two or three, chart and goals stacked
1280px (desktop)	Full navbar, four summary cards in a row, chart beside goals, form beside list

Phone-specific details: buttons and tabs are at least 44px tall so they are easy to tap, form inputs use 16px text so iPhones do not zoom in when one is selected, and the chart drops the "min" unit under each day so the labels fit on one line.

5.3 Accessibility

- Every input has a visible label; error messages are linked to their input with `aria-describedby` .
- All interactive elements have hover and visible keyboard focus styles.
- A "Skip to content" link is the first focusable element.
- The chart includes a hidden data table for screen readers.
- Delete buttons have descriptive labels such as "Delete Morning run (Today)".
- Animations are disabled for users who prefer reduced motion.
- The mobile menu button tells screen readers whether the menu is open (`aria-expanded`).

5.4 Screenshots

All screenshots are taken from the deployed production build with the sample data loaded. Full-page captures of every page at desktop and mobile widths are in the repository under `docs/screenshots/` .

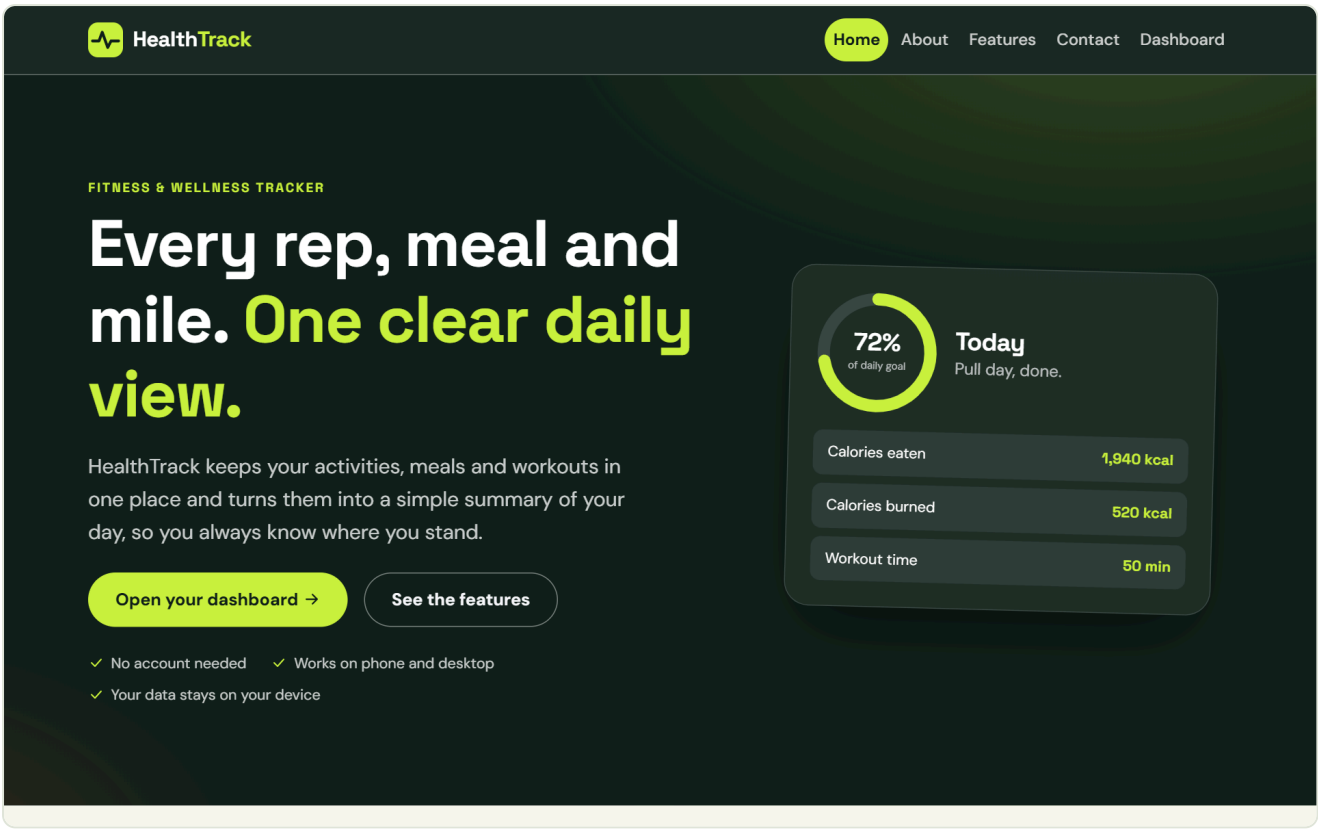


Figure 1. Home page on desktop: hero section with the call-to-action to the dashboard and a preview of the daily summary.

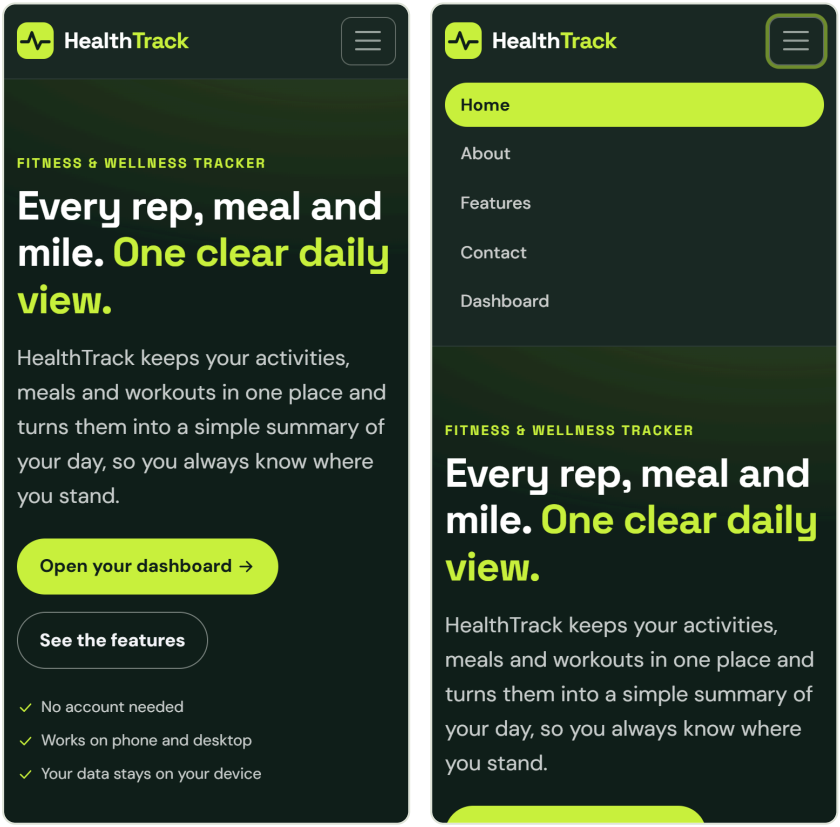


Figure 2. Home page on a phone (375px) and the hamburger menu opened, with the active page highlighted.

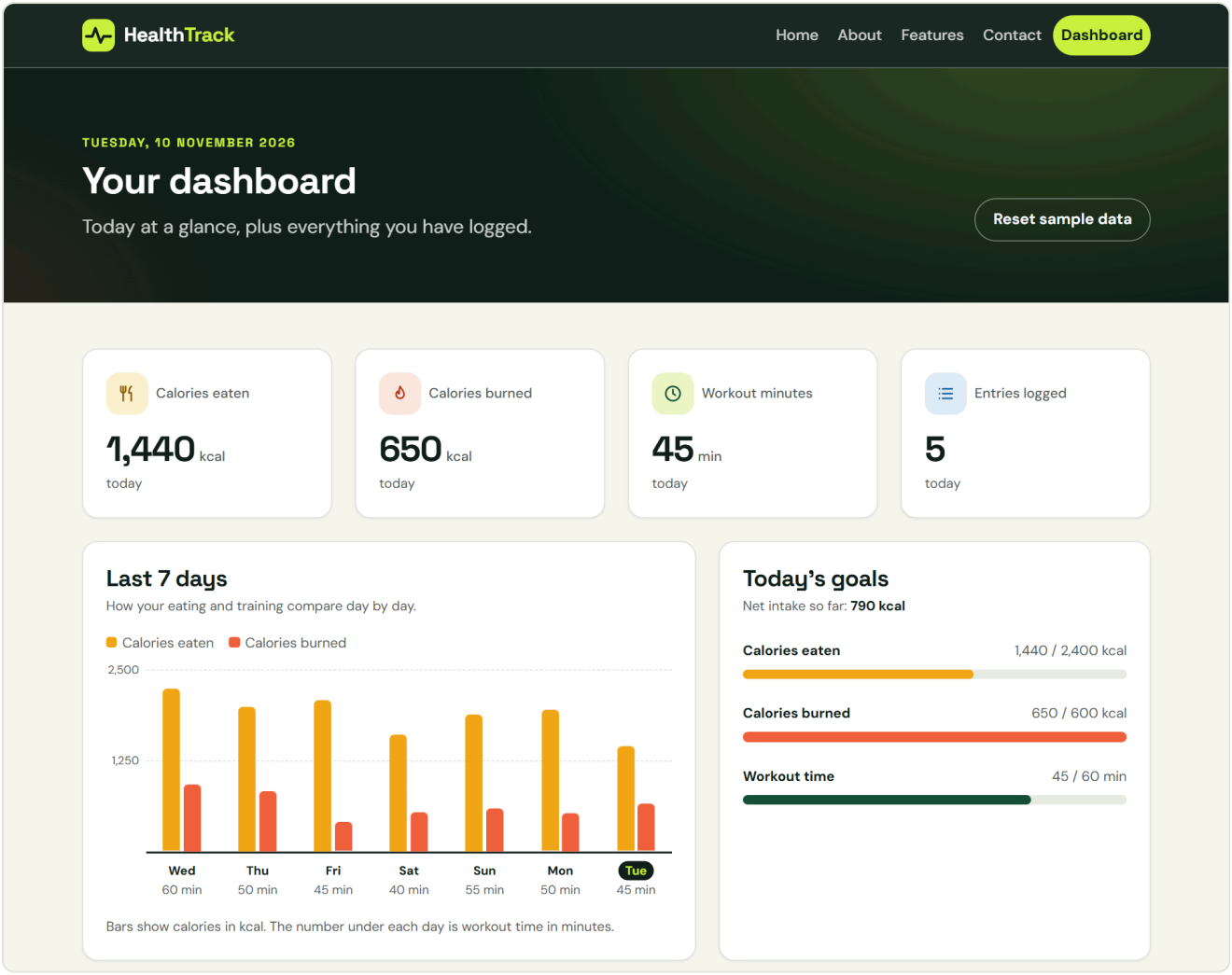


Figure 3. Dashboard on desktop: today's summary cards, the 7-day progress chart built with plain divs, and the daily goal progress bars.

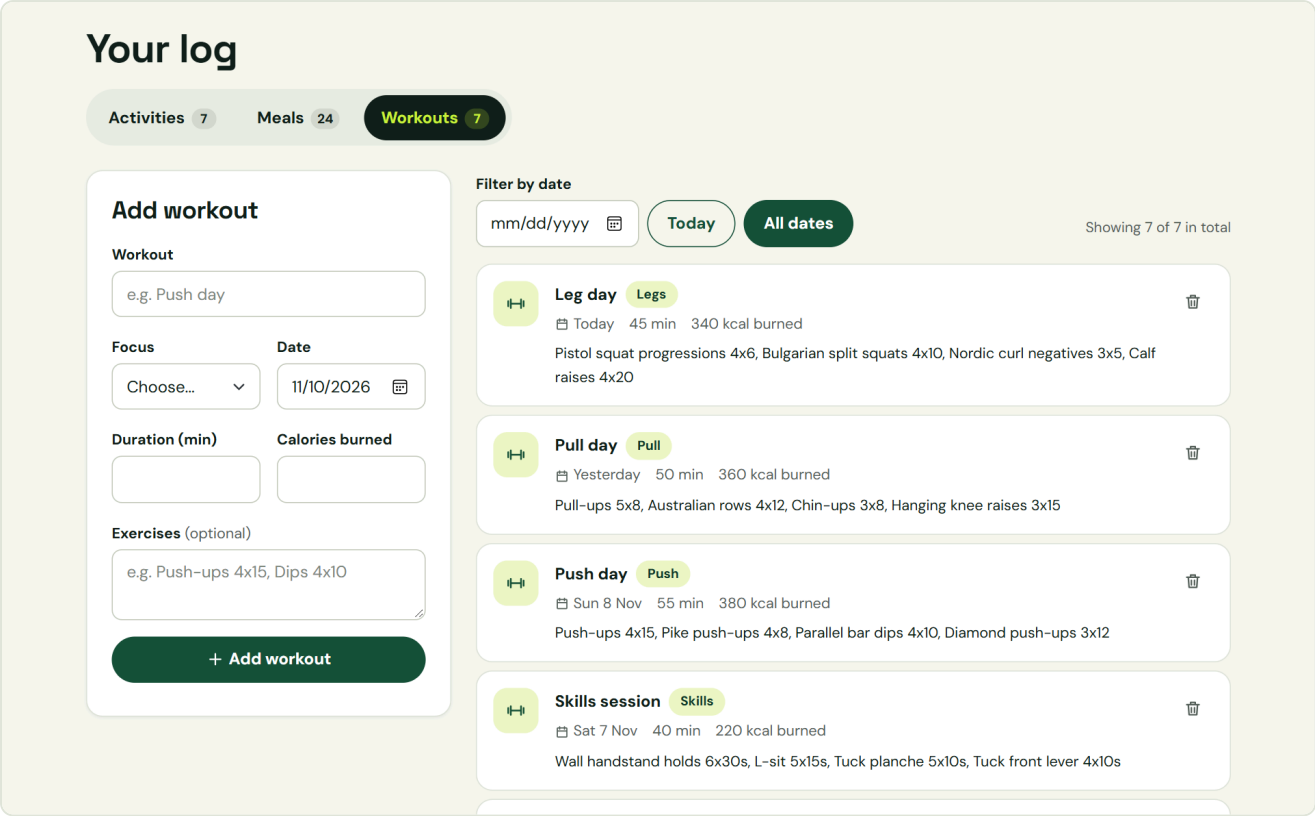


Figure 4. The Workouts tab with the seeded calisthenics push / pull / legs + skills split. The add form is on the left; the list, date filter and delete buttons are on the right.

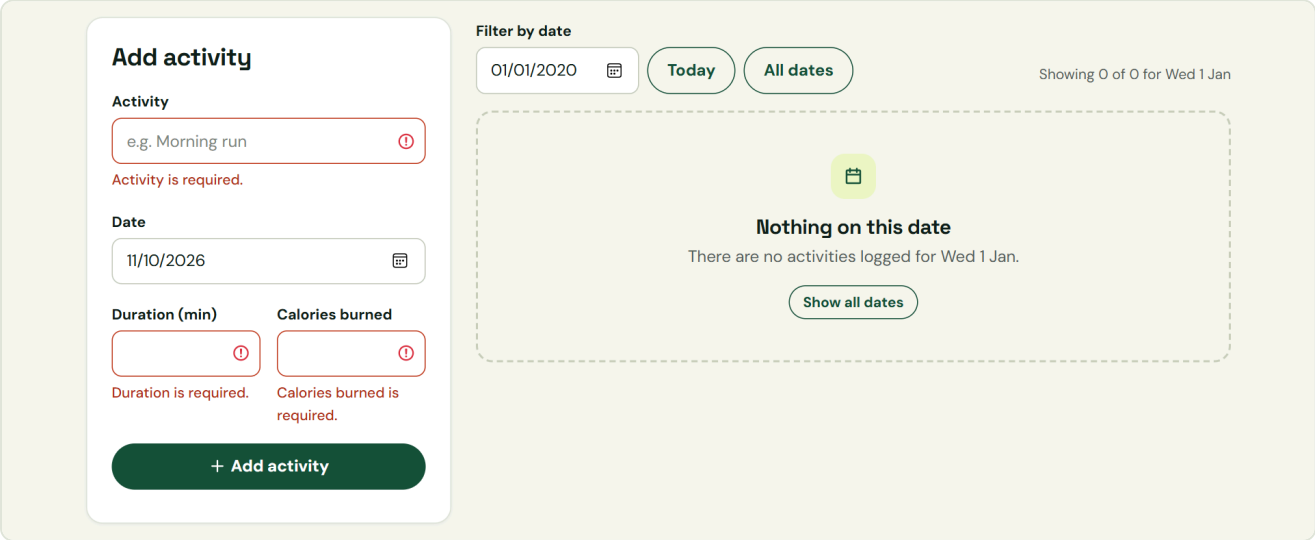


Figure 5. Inline validation after submitting an empty form, and the empty state shown when the date filter matches no entries.

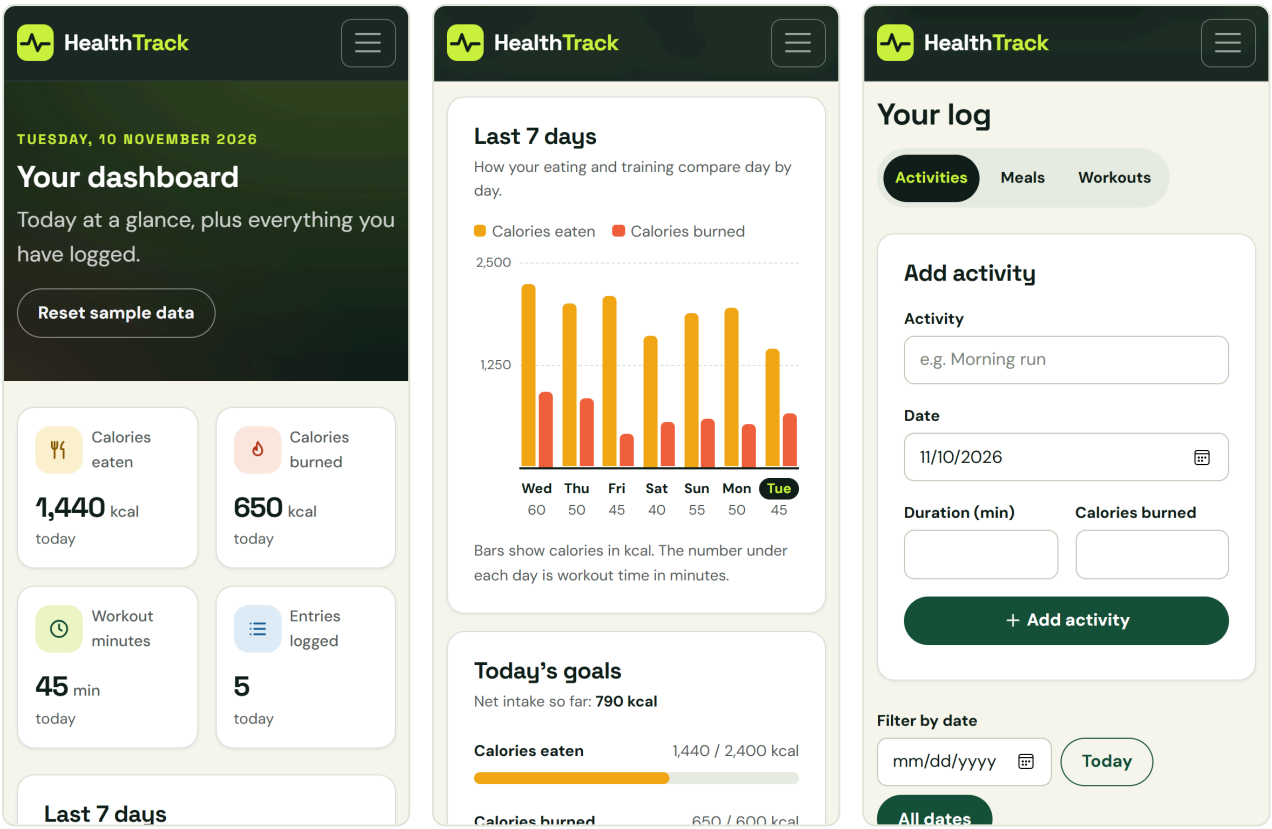


Figure 6. The dashboard on a phone: summary cards two per row, the chart and the tabbed log stacked in a single column.

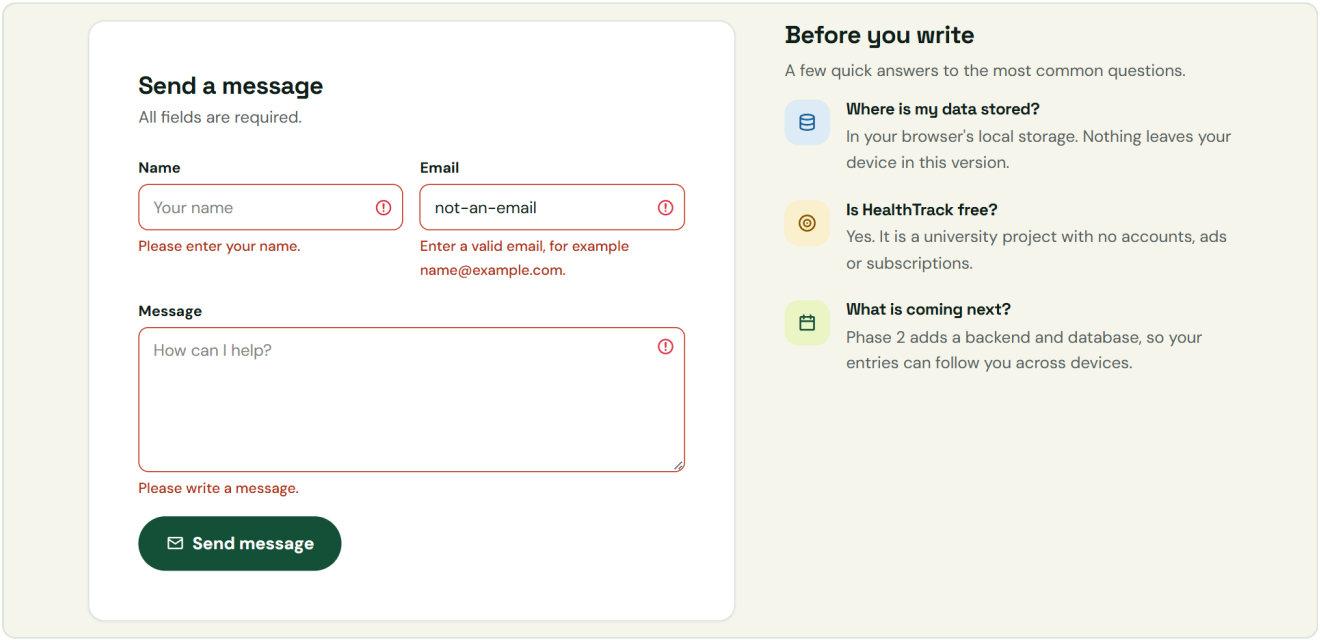


Figure 7. Contact form with client-side validation: each invalid field shows its own message underneath.

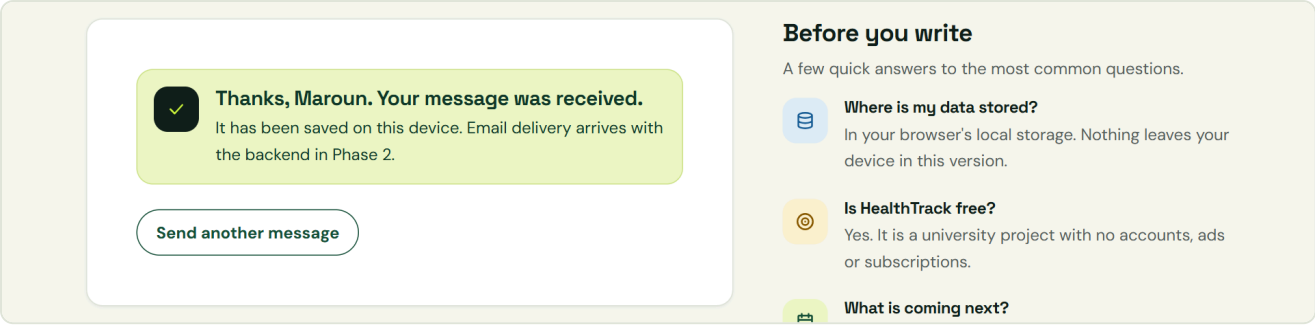


Figure 8. Success state shown after a valid message is submitted.

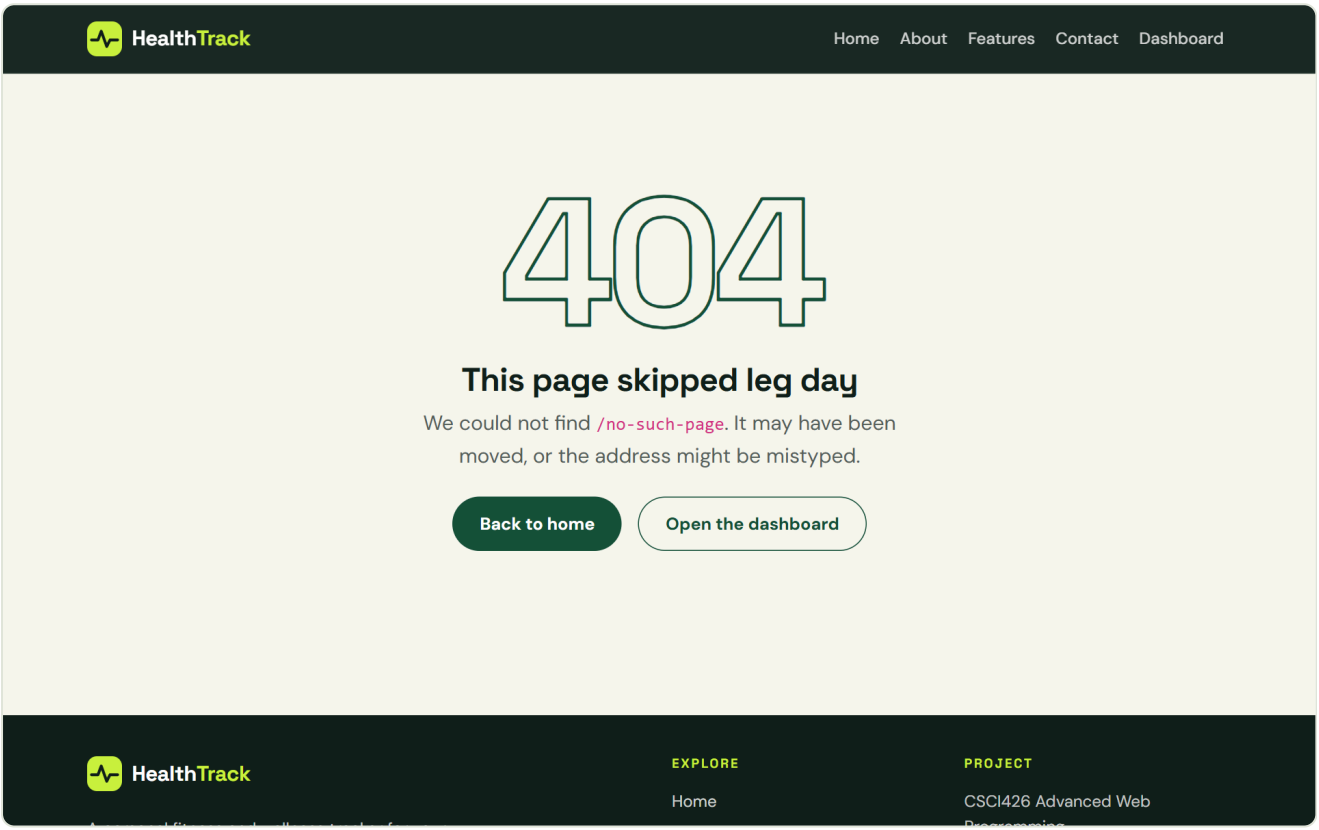


Figure 9. The 404 page shown for any unknown address.

6. Key code snippets

6.1 Routing with a shared layout (`src/App.jsx`)

```
<Routes>
  <Route element={<Layout />}>
    <Route index element={<Home />} />
    <Route path="about" element={<About />} />
    <Route path="features" element={<Features />} />
    <Route path="contact" element={<Contact />} />
    <Route path="dashboard" element={<Dashboard />} />
    <Route path="*" element={<NotFound />} />
  </Route>
</Routes>
```

All pages are children of one `Layout` route. `Layout` renders the navbar, an `<Outlet />` where the matching page appears, and the footer, so the shared parts are written only once. The `*` path catches every unknown address and shows the 404 page.

6.2 Active link highlighting (`src/components/AppNavbar.jsx`)

```
{NAV_LINKS.map((link) => (
  <Nav.Item as="li" key={link.to}>
    <Nav.Link as={NavLink} to={link.to} end={link.to === '/'} eventKey={link.to}>
      {link.label}
    </Nav.Link>
  </Nav.Item>
))}
```

The links come from one array and are rendered with `map`. React Router's `NavLink` adds the class `active` to the link of the current page, which the theme styles as a lime pill. `collapseOnSelect` on the `Navbar` closes the mobile menu after a link is chosen.

6.3 The data service (`src/services/storage.js`)

```
export async function getEntries(type) {
  seedOnFirstLoad()
  return read(type).sort(byNewest)
}

export async function addEntry(type, entry) {
  const created = { ...entry, id: createId(), createdAt: new Date().toISOString() }
  write(type, [created, ...read(type)])
  return created
}

export async function deleteEntry(type, id) {
  write(type, read(type).filter((entry) => entry.id !== id))
}
```

This is the only file that touches `localStorage`. The functions are `async`, so they already return Promises like a network request would. In Phase 2 each body becomes a `fetch()` call (for example `POST /api/meals`) and no page or component needs to change.

6.4 State and derived values (`src/pages/Dashboard.jsx`)

```
const [data, setData] = useState(EMPTY_DATA)

useEffect(() => {
  let ignore = false
  loadAll().then((loaded) => {
    if (!ignore) setData(loaded)
  })
  return () => { ignore = true }
}, [])

const handleAdd = async (type, entry) => {
  const created = await addEntry(type, entry)
  setData((prev) => ({ ...prev, [type]: [created, ...prev[type]] }))
}

const totals = getDailyTotals(data, today)
const week = getWeekSeries(data)
```

The entries are loaded once in `useEffect` and kept in state. Adding an entry saves it through the service and then updates the state with a new object (state is never changed directly). `totals` and `week` are calculated from the state on each render, which is why the summary cards and the chart update as soon as an entry is added or deleted. (Error handling and sorting are omitted here for space.)

6.5 One form for three trackers (`src/data/entryTypes.js` and `src/components/EntryForm.jsx`)

```
meals: {
  label: 'Meals',
  singular: 'meal',
  fields: [
    { name: 'name', label: 'Meal', type: 'text', required: true },
    { name: 'category', label: 'Type', type: 'select', required: true,
      options: ['Breakfast', 'Lunch', 'Dinner', 'Snack'] },
    { name: 'calories', label: 'Calories', type: 'number', required: true, min: 0 },
    // ...
  ],
}
```

```
{fields.map((field) => (
  <Form.Group as={Col} xs={field.half ? 6 : 12} controlId={id} key={field.name}>
    <Form.Label>{field.label}</Form.Label>
    <Form.Control type={field.type} name={field.name} value={values[field.name]}
      onChange={handleChange} isInvalid={Boolean(error)} />
    <Form.Control.Feedback type="invalid">{error}</Form.Control.Feedback>
  </Form.Group>
))}
```

Instead of writing three almost identical forms, each tracker is described by a list of fields. `EntryForm` reads that list and builds the inputs with `map`. The same idea is used by `EntryList`. Adding a new tracker later only needs a new configuration block.

6.6 Validation with inline messages (`src/pages/Contact.jsx`)

```
function validate(values) {
  const errors = {}
  if (!values.name.trim()) errors.name = 'Please enter your name.'
  if (!EMAIL_PATTERN.test(values.email.trim()))
    errors.email = 'Enter a valid email, for example name@example.com.'
  if (values.message.trim().length < 10)
    errors.message = 'Your message must be at least 10 characters.'
  return errors
}
```

`validate` returns an object with one message per invalid field. On submit, the form stores these errors in state; each input shows its message underneath through `Form.Control.Feedback`. If there are no errors, the form is replaced with a success `Alert`. (Shortened: the real function also has separate "required" messages.)

6.7 Filtering and rendering a list (`src/components/TrackerPanel.jsx`)

```
const [filterDate, setFilterDate] = useState('') // '' = all dates
const filtered = filterDate ? entries.filter((entry) => entry.date === filterDate) : entries
```

```
{entries.length === 0 ? (
  <EmptyState title={`No ${config.label.toLowerCase()} yet`} text={config.emptyText} />
) : filtered.length === 0 ? (
  <EmptyState title="Nothing on this date" ... />
) : (
  <EntryList entries={visible} config={config} onDelete={onDelete} />
)}
```

The filtered list is calculated from the entries and the chosen date. Conditional rendering then picks one of three results: an empty state when nothing has been logged, a different empty state when the chosen date has no entries, or the list.

6.8 A chart without a library (`src/components/WeeklyChart.jsx`)

```
const max = niceMax(Math.max(...series.map((day) => Math.max(day.caloriesEaten,
day.caloriesBurned))))
const height = (value) => `${(value / max) * 100}%`

{series.map((day) => (
  <div className="ht-chart-day" key={day.date}>
    <div className="ht-chart-bar is-eaten" style={{ height: height(day.caloriesEaten) }} />
    <div className="ht-chart-bar is-burned" style={{ height: height(day.caloriesBurned) }} />
  </div>
))}
```

Each day is a flex column holding two `div` bars. The height of a bar is its value as a percentage of the largest value in the week (rounded up to a tidy number for the axis). CSS aligns the bars to the bottom and animates them growing.

6.9 Theming Bootstrap with CSS variables (`src/styles/theme.css`)

```
:root {
  --ht-forest: #17503a;
  --ht-lime: #c8f03c;

  --bs-body-bg: var(--ht-paper);
  --bs-body-font-family: var(--ht-font-body);
  --bs-primary: var(--ht-forest);
  --bs-border-radius: var(--ht-radius-sm);
}

.btn-primary {
  --bs-btn-bg: var(--ht-forest);
  --bs-btn-hover-bg: var(--ht-forest-dark);
}
```

Bootstrap 5 reads its colours, fonts and radii from CSS variables. The theme first defines the project's own tokens (`--ht-*`) and then assigns them to Bootstrap's variables (`--bs-*`). Because `theme.css` is imported after Bootstrap in `main.jsx` , these values win and every Bootstrap component picks up the HealthTrack look.

7. Testing

Check	Result
`npm run lint`	No errors or warnings
`npm run build`	Builds successfully
Browser console on every page	No errors or warnings
Layout at 375px, 768px and 1280px	No horizontal scrolling on any page

Check	Result
Phone emulation from 320px to 414px and landscape	Menu, tabs, forms, add and delete all work by touch
Deployed site on Vercel	Every route loads directly and after a refresh
Add, delete and filter entries	Lists, summary cards and chart update correctly
Data after page refresh	Entries are still there
Contact form	Shows errors for empty and invalid fields, then the success state
Unknown address	Shows the 404 page

8. Phase 2 plan

- Build a REST API (for example Node.js and Express) with endpoints for activities, meals, workouts and contact messages.
- Store the data in a database, one table per entry type.
- Replace the function bodies in `src/services/storage.js` with `fetch()` calls.
- Add user accounts so each person sees only their own data.

9. Conclusion

Phase 1 delivers a complete, responsive React frontend for HealthTrack with six routes, a dynamic dashboard and its own visual identity. Keeping all data access in one service file and all entries in one piece of state made the dashboard simple to reason about and leaves a clear path for adding the backend in Phase 2.

10. Group contribution statement

This project was completed individually. The group has one member.

Member	Student ID	Contribution
Maroun Issa	52430635	100%: planning, visual design, all pages and components, data layer, responsive layout and testing, documentation and report, GitHub repository and Vercel deployment

11. References

- React documentation: https://react.dev/
- React Router documentation: https://reactrouter.com/
- React Bootstrap documentation: https://react-bootstrap.github.io/
- Bootstrap 5 documentation: https://getbootstrap.com/docs/5.3/
- Vite documentation: https://vite.dev/
- Vercel documentation: https://vercel.com/docs